Fuzz Testing

Software Testing (COM3529)

Checkin code: RW-MV-EE

José Rojas & Neil Walkinshaw

Week 10

Roadmap

Beizer’s Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

The origin story of Fuzzing

Late 80s, on a dark stormy night in Wisconsin

Bart Miller (U Wisc.) logged into work over a dial-up.

- There was a lot of line noise due to the storm.
- His shell and editors kept crashing.

Idea:

- Feed “fuzz” - streams of pure randomness to OS and utility code. E.g., noise from `/dev/random`
- Watch them break!
- This would crash 25-33% of utilities.

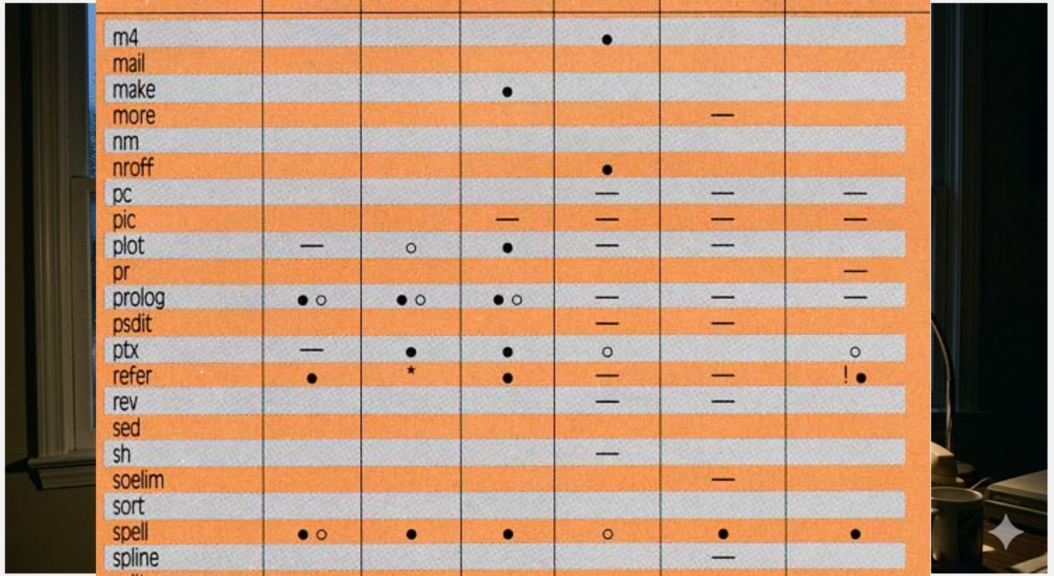


TABLE II. List of Utilities Tested and the Systems on which They Were Tested (part 2)

*•=utility crashed, ○=utility hung, *=crashed on SunOS 3.2 but not on SunOS 4.0, ⊕ = crashed only on SunOS 4.0, not 3.2. —=utility unavailable on that system. !=utility caused the operating system to crash.*

Utility	VAX (v)	Sun (s)	HP (h)	i386 (x)	AIX 1.1 (a)	Sequent (d)
m4				•		
mail						
make			•			
more					—	
nm						
nroff				•		
pc				—	—	—
pic			—	—	—	—
plot	—	○	•	—	—	
pr						—
prolog	• ○	• ○	• ○	—	—	—
psdit				—	—	
ptx	—	•	•	○		○
refer	•	*	•	—	—	! •
rev				—	—	
sed						
sh				—		
soelim					—	
sort						
spell	• ○	•	•	○	•	•
spline					—	
split						
sql		—			—	—
strings					—	
strip						
style	•	—	•		—	•
sum						
tail						
tbl						
tee						
telnet	•	•	•	—	•	○
tex			—	—	—	—
tr						
troff	—	—	—			
tsort	•	*	•	•	•	•
ul	•	•	•	—	—	•
uniq	•	•	•	•	•	•
units	• ○	•	•	•	•	•
vgrind	•		—	—	—	
vi	•		•	—		
wc						
yacc						
# tested	85	83	75	55	49	73
# crashed/hung	25	21	25	16	12	19
%	29.4%	25.3%	33.3%	29.1%	24.5%	26.0%

Miller, Barton P., Lars Fredriksen, and Bryan So. "An empirical study of the reliability of UNIX utilities." C

Rationale

Software tends to be developed and tested according to the "happy path".

- Expected / typical inputs.

Fuzzing will explore the "unhappy path".

- Unexpected, quasi-random inputs.
- Can check software in terms of its robustness - its ability to handle unexpected inputs.
- Can guard against (or at least identify) potential security vulnerabilities.

It's (relatively) easy.

- Generating garbled input.
- Running automatically.
- Checking if the program has crashed.

Extremely popular in industry

- Google's ClusterFuzz platform scales fuzz testing to multiple VMs.
  - Their OSS-Fuzz instance runs on >>100,000 VMs continuously.
  - Has identified >13,000 vulnerabilities and 50,000 bugs across 1,000 OS projects.
  - Identified >27,000 bugs within the Google ecosystem (e.g. Chrome).
- Routinely used within Apple, Microsoft, Mozilla, RedHat, Amazon, Meta, etc.
- Increasingly used to test cyber-physical systems.
  - Fuzz testing ICs, self-driving cars, smart manufacturing systems, etc.

The Fuzzing Process

The work-flow

Input generation

Pure random:
Use a random noise generator / select from a random distribution.

Mutation-based:
Make quasi-random "tweaks" to valid inputs.

Generation-based:
Generate quasi-random inputs according to an input specification, e.g. an input grammar.

Execution

Black-box:
We can only monitor the outputs, no possibility of monitoring internal coverage.

White-box:
Monitor internal characteristics such as coverage, which can inform input generation.

Defect monitoring

Crash detection:
Monitor the application at an OS level to detect if it has crashed.

Existing test oracles:
If they exist - ensure that no assertions are violated.

Mutation-based Fuzzing

Imagine your target is one of the following:

- A Java compiler.
- The Chrome web browser.
- A JPEG rendering engine.

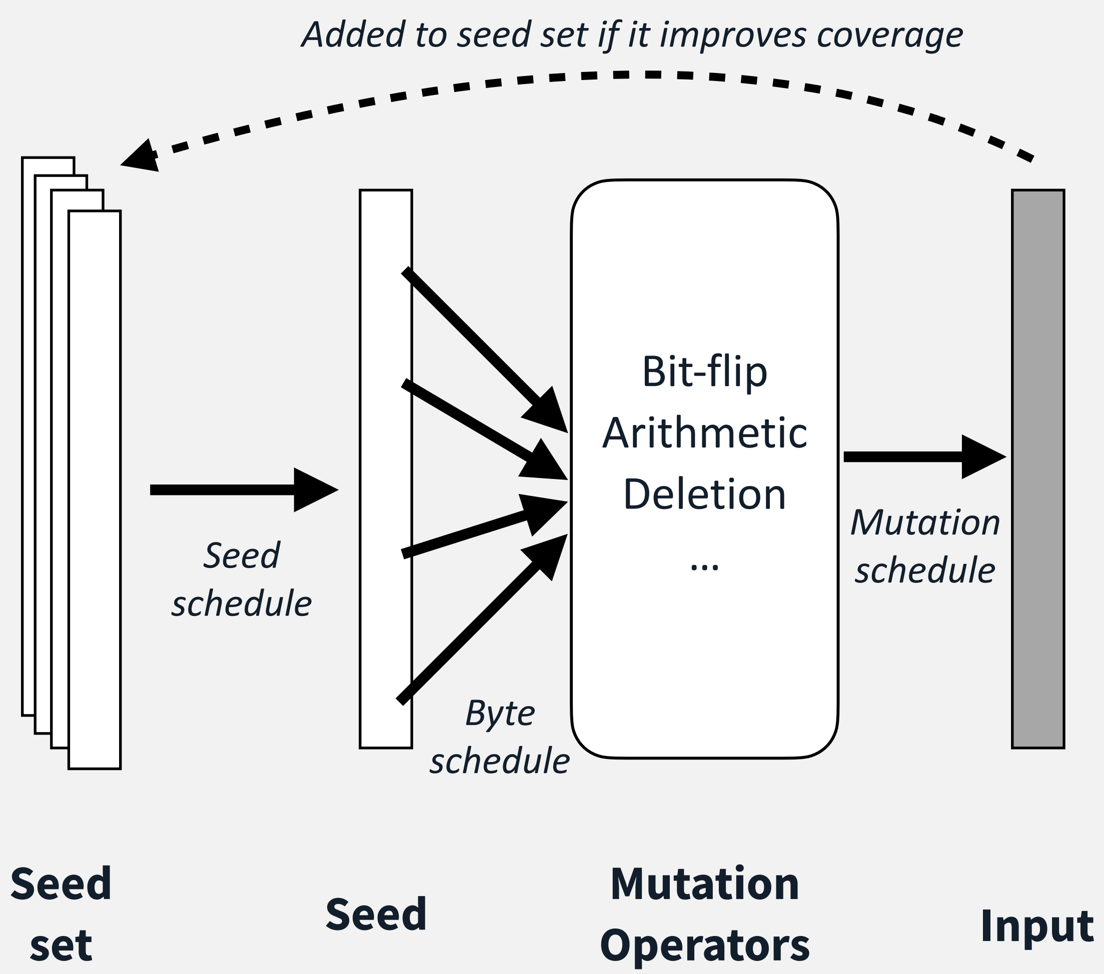
Purely random inputs will only explore a fraction of the target.

Idea: Start from existing valid inputs (seeds), and randomly modify (mutate) them.

Seed schedule - frequency of picking a specific seed.

Byte schedule - frequency by which a given byte / element within the seed should be mutated.

Mutation schedule - frequency by which a given mutation operator should be applied.



Grammar-based Fuzzing (an example of Generation-based Fuzzing)

Instead of starting from seeds, let us suppose we have a specification that describes the input format.

Commonly specified as a Context-Free Grammar.

```
<start> ::= <person_name> "," <age>
<person_name> ::= <first_name> " " <last_name>
<first_name> ::= <name>
<last_name> ::= <name>
<name> ::= <ascii_uppercase_letter><ascii_lowercase_letter>+
<age> ::= <digit>+
```

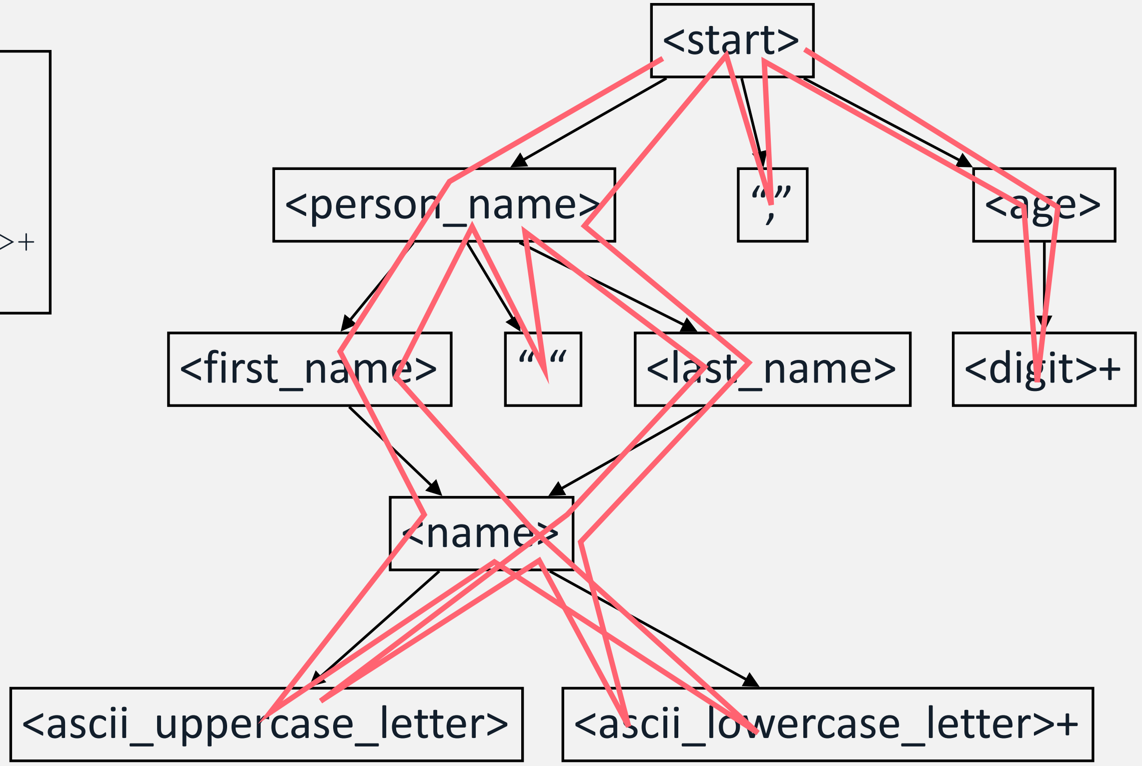
```
Vedfxeuqddtmhu Irmwggurhntxmupibfwl,2199401311112419745

Hddpocedjpicxykycpjz Zswswfromrpstrzlvoq,84089665816913
Okhcemauculgfyzrcdjr Flyclqtbkaciudaeo,75003115414884861

Gmfgnipeisxv Eggqbtztkgbshawro,032776289999332
Cjfjnwjxrxwumhxrytqfg Akiu,519682499
Anei Bamdhn,36734

Dxqhwuvqs Hd,9
Nigtklbvnwhgtaskvtt Ibujxujfgweglth,20190840

Rjnyqxclpvonxeqyy Pylfcbrs,43539303502100818
Vikwmawlphilsiq Oxydjpsscsyqita,10540608862298442
```



White-box Fuzzing

Exploiting Coverage

- Exploit feedback from runtime to improve input selection / generation.
  - Track the branch coverage.
  - Can also use other forms of coverage, depending on the nature of the target system.
    - E.g. coverage of a grammar.
- Identical purpose to Search-Based Software Testing.
  - In a fuzzer, feedback can be used to adapt **seed schedule, byte schedule, mutation schedule**.
- Under the hood, fuzzers often employ Genetic Algorithms to inform this process.

Finding Specific Input Conditions

Specific paths through the code require very specific conditions on inputs.

- Randomly generating / mutating inputs generally fails to find these.

Numerous white-box code analysis techniques exist to enable this.

- **Dynamic Taint Analysis**
  - Include a special ‘marker’ in the input, and record where it crops up in the source code (e.g. which variables contain the marker at which points).
  - Can be used to establish relationships between input components and statements in code.
  - *Hard to implement* - requires ability to extract lots of runtime information at each execution step.
- **Symbolic execution**
  - Turn conditionals in source code (e.g. if-predicates) into expressions to be solved by constraint solvers.
  - Restrict seed generation / selection to those that satisfy the constraints.
  - *Constraint solvers can be time and resource-consuming, cannot always solve constraints usefully.*
- **Input format inference**
  - Use Machine Learners to infer format of inputs from seed set. Use to focus seed generation.
  - *Requires lots of data, do not always provide helpful input formats.*

Key limitations

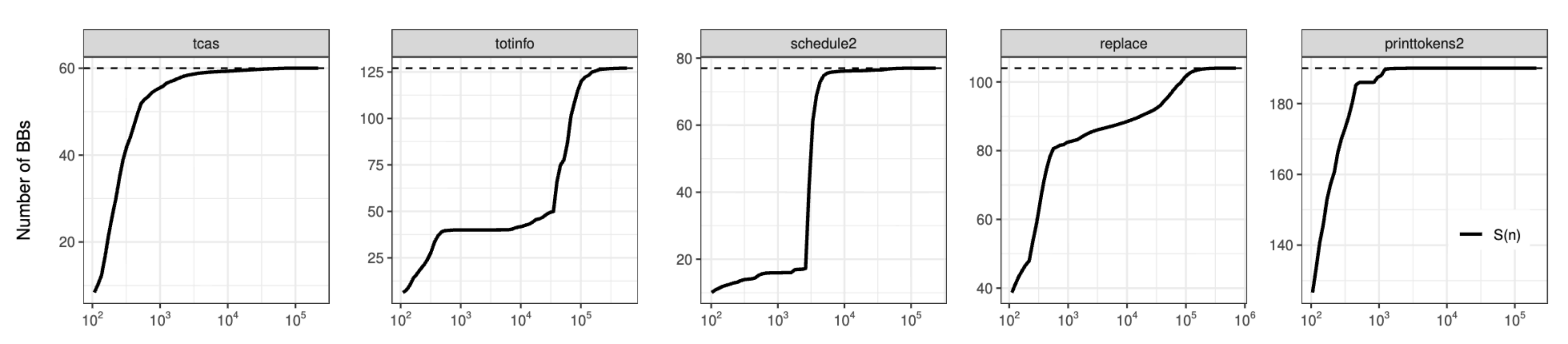
When to stop?

Fuzzing consumes a significant amount of compute-time.

- Fuzzing campaigns are often measured in CPU-months.

Ideally want to finish testing when a “saturation point” has been reached.

- I.e. coverage stops increasing.
- There are no criteria that can be safely used to establish this.
- Fuzzers often run indefinitely.



Liyanage, Danushka, et al. "Reachable coverage: Estimating saturation in fuzzing." 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023.

Dependence on seeds or specs

For mutation-based fuzzing:

- Effectiveness determined by seed set.
- A “richer” set of seeds will lead to more effective test set.
- May be hard to obtain, depending on the type of software.

For generation-based fuzzing:

- Grammars tend to be widely available…
- … but may only consider “happy-path” inputs.
- Or may be incomplete.
- Obtaining an ideally fuzzable spec can require a significant manual effort.

Measuring “fitness” and the Oracle Problem

White-box fuzzing is based on maximising coverage.

- As with Search-Based Testing (and other forms of testing) coverage can be misleading.
- *“Once a measure becomes a target, it ceases to be a good measure”. [Goodhart’s law]*

Crashes are a very coarse indicator of whether a program has a fault.

- Just because it doesn’t crash doesn't mean it is “correct”.
- The oracle problem persists.

Good starter tools

AFL++ (American Fuzzy Lop ++) - https://aflplus.plus/

- The de-facto fuzzer, widely used.
- Mutation-based, coverage-driven, with support for test-case / corpus minimisation.

Fandango - https://github.com/fandango-fuzzer/fandango

- A grammar-based fuzzer.